Pipeline Pulse — Project Plan

Oct 30, 2026 · @Someone

Overview

Pipeline Pulse is a 6-week portfolio project: an event-driven engine that predicts which B2B deals will close or slip, and an AI agent that explains why and suggests the next action. It is built to pitch myself to Outreach leadership for the Software Engineer (IIT campus) role.

The problem it targets. Outreach is a Gartner Leader in Revenue Action Orchestration, but forecasting is its clearest gap. Its closest rival, Salesloft, merged with Clari in December 2025 to add forecasting, and reviewers note that Outreach customers typically pair it with a separate forecasting tool (Artisan, Sep 2026).

The pitch in one line. "Teams often buy a second tool for forecasting. Here is a working prototype of how Outreach could own that workflow with its own activity data and agents."

How it is built. Real public deal data is the backbone. A simulator I design adds the realistic activity stream (emails, replies, meetings, stage changes) that public data lacks. Go services consume the stream, a Python model scores deals, and an LLM agent exposed over MCP explains them — mirroring Outreach's own stack (Go, Kafka, AWS, Kubernetes, React/TypeScript).

Goals, non-goals and success metrics

The project succeeds if a reviewer can run it with one command, see a forecast that beats a stage-weighted baseline, and read a design doc that justifies every major choice.

Goals

1. Ingest real public deal data and enrich it with a calibrated, outcome-linked activity simulator.
2. Process activity as a stream and keep a live health record per deal.
3. Predict close probability and slip risk per deal, and roll up a team forecast with a confidence range.
4. Explain each at-risk deal and suggest a next action through an LLM agent, exposed as MCP tools.
5. Ship it production-style: containers, CI, tests, metrics, and written design decisions.

Non-goals

- Sending real emails or building a sequencer — Outreach already does that well.
- Claiming results on real Outreach customer data, which I do not have.
- A polished multi-tenant SaaS: one tenant, local or single-cluster deployment.

Success metrics

Area	Metric	Target
Model	Brier score vs stage-weighted baseline	At least 15% lower
Model	AUC on a time-based holdout	0.75 or higher
Model	Calibration error (ECE)	Under 0.05
Early warning	Lost deals flagged at-risk 14+ days before close	60% or more
Pipeline	Sustained throughput	1M events/hour on a laptop
Pipeline	Event-to-score latency (p95)	Under 2 s
API	Read latency (p95)	Under 100 ms
Agent	Explanations grounded in real deal facts (manual check of 50)	90% or more
Ops	Setup for a reviewer	`make up` in under 10 minutes

The model targets are starting guesses; adjust them after the first baseline run and report the real numbers either way.

Data strategy

Real public data supplies the deals and their outcomes; the simulator supplies the activity stream that no public dataset has. Outreach's own customer data is confidential and out of reach, so the project says so openly and mirrors Outreach's public API objects instead.

Source	What it gives	Role in the project
Maven Analytics — CRM Sales Opportunities	~8,800 B2B opportunities with accounts, products, sales agents, stages (Prospecting → Engaging → Won/Lost), dates and values	Deal backbone and ground-truth outcomes; calibration targets for cycle length and win rate

Source	What it gives	Role in the project
Olist — Marketing Funnel (Kaggle)	~8k real anonymised leads (Jun 2017–Jun 2018), ~840 closed deals with SDR/SR ids, segment, lead type, behaviour profile	Real lead-to-deal conversion and lead-source mix; second holdout to test generalisation
Enron email corpus (CMU)	~500k real business emails	Reply-latency and thread-length distributions; realistic text templates for the agent
Salesforce Developer Edition (free org)	A real CRM with an API	Load Maven + Olist deals; the Go connector pulls them through the real Salesforce API
HubSpot developer test account (free)	A second CRM	Optional second connector — HubSpot sync is a frequent Outreach complaint
Outreach Developer Portal (api.outreach.io, OAuth 2.0)	Public API object model	Try to get sandbox access; if not, mirror its objects (prospects, accounts, opportunities, mailings, sequences) in the event schema

The real → simulator pipeline

1. Load and clean Maven and Olist into a common `deals` table (id, account, owner, stage history, amount, created, closed, outcome).
2. Fit distributions from the real data: cycle length by product and outcome, win rate by segment and owner, deal size.
3. Fit communication distributions from Enron: reply delay, thread length, time-of-day patterns.
4. For each real deal, the simulator generates an activity timeline consistent with its real outcome and dates.
5. Validate: the simulated data must reproduce the real aggregates (see Simulator design).
6. Train on it and report results two ways — real-only features and real + simulated features — so it is clear what the simulator adds.

Honesty line for the pitch: "Built on public B2B data with a calibrated synthetic activity layer; the schema mirrors Outreach's API, so it is ready for real customer data."

Simulator design

The simulator turns each real deal into a believable activity timeline by driving event rates from a hidden "deal health" that drifts toward the deal's real outcome — with enough noise that no single signal gives the answer away.

Entities

- **Account** — from Maven/Olist: industry, size, region.
- **Deal** — from Maven/Olist: owner, product, amount, created date, close date, outcome.
- **Contacts** — generated: 1–6 buyer-side people per deal with roles (champion, economic buyer, technical evaluator, end user).
- **Rep** — from the real owner/agent ids, each with a generated skill level and response habit.

Event types

Event	Key fields	Driven by
`email_sent`	deal, rep, contact, sequence step, template id	Rep cadence, stage
`email_opened` / `link_clicked`	message id, delay	Health, contact role
`email_replied`	message id, delay, sentiment (pos/neutral/neg)	Health; delay fitted from Enron
`call_logged`	duration, connected yes/no	Stage, rep habit
`meeting_scheduled` / `meeting_held` / `meeting_no_show`	attendees, type (discovery, demo, pricing, legal)	Stage, health
`contact_added`	new contact role	Health (multithreading)
`stage_changed`	from, to	Real stage history from the dataset
`close_date_changed`	old, new	Health falling near the close date
`deal_closed`	won/lost, amount	Real outcome

Generative process (per deal)

1. Read the real deal: created date, close date, outcome, amount, owner.
2. Draw a latent health path h(t) in [0, 1]: start near 0.5 and drift toward a won target (around 0.8) or a lost target (around 0.2), with random-walk noise. Some won deals dip and recover; some lost deals look healthy until late.
3. Each event type is a Poisson process whose rate depends on stage, rep habit and h(t). Replies, meetings held and new contacts rise with health; no-shows, reply delays and close-date pushes rise as health falls.
4. Place real stage changes on their real dates; meetings of the matching type cluster around them (demo before Engaging, pricing before close).
5. Stop all activity at the real close date and emit `deal_closed` with the real outcome.

Outcome-linked patterns (probabilistic, never deterministic)

Pattern	Won deals	Lost deals
Buyer-side contacts	Usually 3 or more by the late stage	Often 1 (single-threaded)
Reply delay trend	Stable or shortening	Lengthening in the final weeks
Silence after a pricing meeting	Rare	Common (10+ days)
Close-date pushes	0–1	2 or more
Economic buyer in a meeting	Likely	Unlikely
Meeting no-shows	Rare	Rising toward the close

Inject noise on purpose: about 15% of deals break these patterns. Without that the model scores near-perfectly and the result means nothing.

Calibration and validation

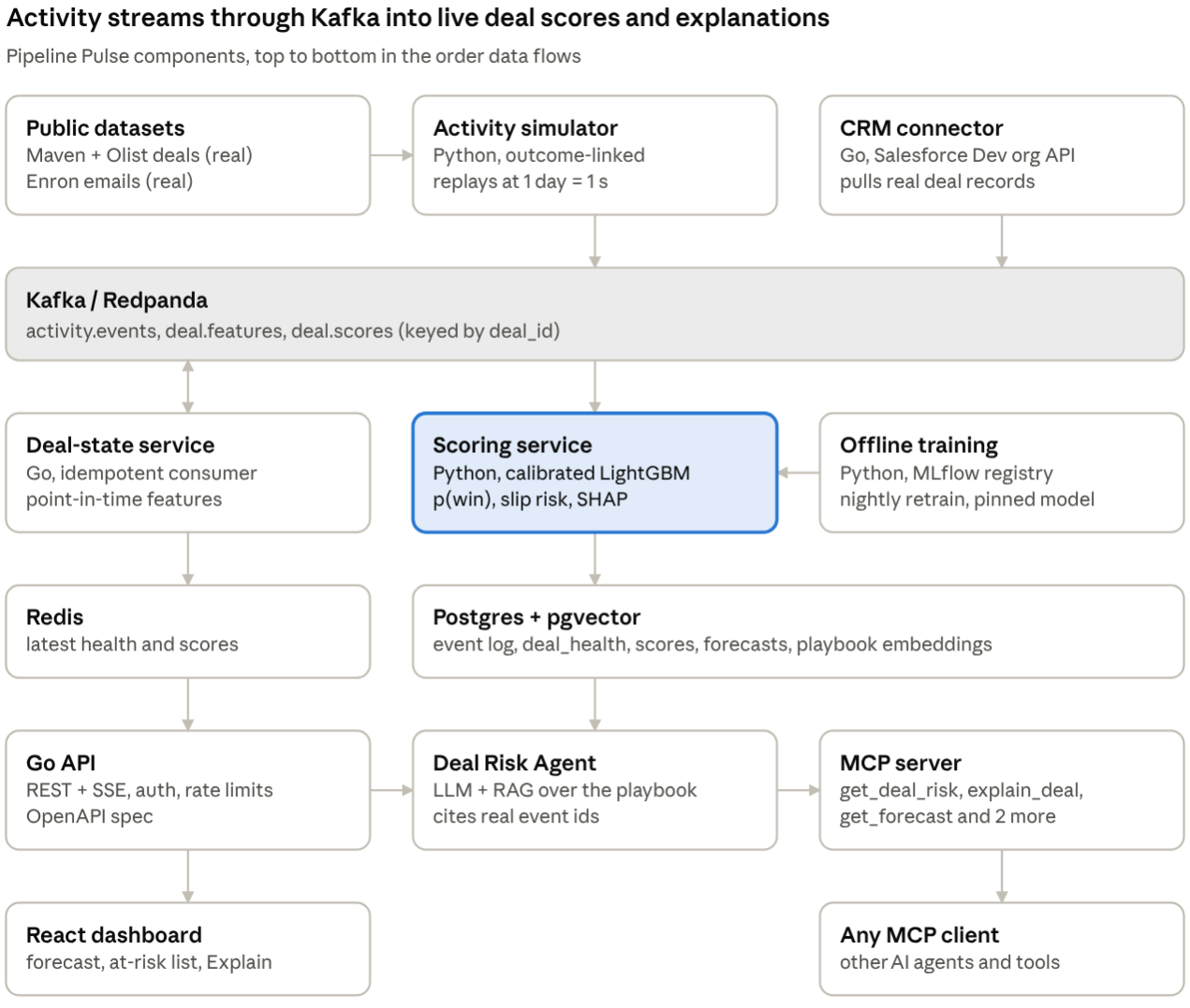
- Real aggregates must match within about 5%: win rate, cycle length by outcome, deal-size distribution, deals per rep.
- Communication shape must match Enron: the reply-delay distribution (compare with a KS test) and the time-of-day/weekday mix.
- **Leakage check:** train a model on events before day X only; if it scores near-perfectly, the patterns are too strong — add noise.
- **Recovery check:** plant a known effect (for example multithreading), then confirm the model's feature importance finds it.

Modes and config

- **Batch:** writes a Parquet/CSV event log for training.
- **Stream:** replays events into Kafka in time order with a speed-up factor (1 day = 1 second) for the live demo and load tests.
- One YAML config (seed, noise level, pattern strengths, speed-up) so every run is reproducible.

System architecture

Data flows one way: real deals and simulated activity enter Kafka, Go services keep live deal state, a Python service scores each change, and the agent and API serve the results.



Pipeline Pulse architecture · 12 components

The scoring service is the core; everything above it feeds it events and everything below it serves its scores.

Component	Language / tech	Why this choice
Simulator, ML, scoring	Python (pandas, LightGBM, SHAP, MLflow)	Best ML ecosystem; matches the JD's Python ask
Connector, deal-state, API	Go	Outreach's main backend language for new services
Event bus	Kafka (Redpanda locally)	Ordered per-deal streams, replay, matches Outreach's async stack
Storage	Postgres + pgvector, Redis	One database for relational data and embeddings; Redis for hot reads
Frontend	React + TypeScript	Matches Outreach's frontend
Infra	Docker, Kubernetes (Helm), Terraform, Prometheus/Grafana	Matches Outreach's infrastructure stack

Data model and event schemas

Every event shares one envelope and is keyed by `deal_id`, so all events for a deal land on the same Kafka partition and are processed in order.

Event envelope (Protobuf, or JSON Schema for v1)

```
{
  "event_id": "uuid",
  "event_type": "email_replied",
  "occurred_at": "2026-03-14T10:22:05Z",
  "tenant_id": "demo",
  "deal_id": "opp_1042",
  "account_id": "acc_88",
  "actor": {"kind": "contact", "id": "con_311", "role": "economic_buyer"},
  "payload": {"message_id": "msg_77", "reply_delay_hours": 5.2, "sentiment":
"positive"},
  "schema_version": 1
}
```

Kafka topics

Topic	Key	Producer	Consumers
`activity.events`	deal_id	Simulator, CRM connector	Deal-state service
`deal.features`	deal_id	Deal-state service	Scoring service
`deal.scores`	deal_id	Scoring service	API (cache refresh), alert worker
`activity.events.dlq`	deal_id	Any consumer	Manual replay tool

Postgres tables

Table	Purpose	Key columns
`accounts`	Account master	id, name, industry, size, region
`deals`	Deal master from the CRM	id, account_id, owner_id, amount, stage, created_at, expected_close, outcome
`contacts`	Buyer-side people	id, deal_id, role, added_at
`events`	Append-only event log (partitioned by month)	event_id (unique), deal_id, type, occurred_at, payload jsonb
`deal_health`	Live feature snapshot per deal	deal_id, features jsonb, updated_at, last_event_id
`deal_scores`	Score history	deal_id, scored_at, p_win, slip_risk, model_version, top_factors jsonb
`forecasts`	Team roll-ups	team_id, period, expected, p10, p90, generated_at
`processed_events`	Idempotency guard	event_id, processed_at

Redis caches the latest score and health per deal for the dashboard and MCP reads.

Design rules

- **Idempotent consumers:** check `event_id` against `processed_events` in the same transaction as the update, so replays and redeliveries are safe.
- **Out-of-order events:** features are computed from event time, not arrival time; late events trigger a recompute for that deal.
- **Schema evolution:** add fields only; bump `schema_version` for breaking changes.

ML: features, models, evaluation

A calibrated gradient-boosted model predicts each open deal's win probability and slip risk from point-in-time features, and must beat the stage-weighted baseline that most sales teams use today.

Features (computed as of each scoring time — never using future events)

Group	Examples
Engagement	Emails sent/replied (7/14/30-day windows), reply rate, meetings held vs scheduled
Momentum	Trend in reply delay, days since last buyer reply, activity slope week over week
Multithreading	Buyer contacts engaged, economic buyer involved yes/no, roles covered
Stage dynamics	Days in current stage vs the median for that product, stage regressions
Risk signals	Close-date pushes, no-shows, days of silence after a pricing meeting
Deal context	Amount (log), product, industry, account size, rep's historical win rate (from training data only)

Models

1. **Baseline:** stage-weighted probability (historical win rate per stage) — the number to beat.
2. **Logistic regression** on the same features — simple and interpretable.
3. **LightGBM classifier** for p(win) — the main model, calibrated with isotonic regression.
4. **Slip risk:** a second LightGBM for p(close date moves out of the current quarter); optionally a survival model (lifelines) for time-to-close as a stretch goal.
5. **Team forecast:** Monte Carlo over open deals (each won with its p(win), closing in-period with its timing estimate) to give expected bookings with P10/P90 bands.

Evaluation

- **Time-based split:** train on deals created before a cutoff, test on later deals. Never split randomly — that leaks future patterns.
- **Snapshot scoring:** score each test deal at 25/50/75% of its life to show how early the model is useful.

- **Metrics:** AUC, Brier score, calibration curve and ECE, and early-warning recall (share of lost deals flagged 14+ days before close).
- **Forecast accuracy:** MAPE of the predicted vs actual quarterly bookings, and P10–P90 coverage.
- **Ablations:** real-only features vs real + simulated, and with/without multithreading features.
- **Explainability:** SHAP values per prediction; the top 3 factors feed the agent.

MLOps (lightweight)

- MLflow for experiment tracking and the model registry.
- The scoring service loads a pinned model version; every score row stores `model_version`.
- A nightly retrain job, plus a drift report comparing feature distributions (Evidently or a simple PSI script).

Agent layer: Deal Risk Agent, RAG, MCP tools

The agent turns a score into a sentence a rep can act on: why this deal is at risk, citing real events, and one concrete next step drawn from a sales playbook.

How a request flows

1. The user (or another agent) asks "Why is the Acme deal at risk?"
2. The agent calls tools to fetch the deal's score, top SHAP factors and its last 30 days of events.
3. It retrieves 2–3 relevant playbook passages (for example "single-threaded late-stage deals") from a vector store.
4. The LLM writes the explanation and next step, and must cite event ids and playbook passages it used.
5. A guard checks every cited fact exists in the fetched data; if not, it retries once or returns the facts without prose.

Knowledge base for RAG

- A short, self-written sales playbook (10–15 pages in Markdown): multithreading, re-engaging silent deals, handling pricing objections, close-date hygiene.
- Chunked, embedded and stored in pgvector inside the same Postgres, so there is no extra database.
- This mirrors Outreach Knowledge, which grounds AI answers in company-approved content.

MCP server tools

Tool	Input	Output
`get_deal_risk`	deal_id	p_win, slip_risk, top factors, last updated
`explain_deal`	deal_id	Grounded explanation with cited events
`suggest_next_action`	deal_id	One action, reason, a draft email if relevant
`get_forecast`	team_id, period	Expected bookings, P10/P90, at-risk deals
`list_at_risk_deals`	team_id, limit	Ranked list with one-line reasons

Outreach has launched its own MCP support for agent-to-agent workflows, so these tools show how Pipeline Pulse could plug into that ecosystem.

Model and evaluation

- Any API LLM works; keep a provider interface so a small open model (via Ollama) can run offline.
- Build a 50-case eval set: deal snapshot plus the expected key reasons. Score groundedness (cited facts are real), relevance (matches top factors) and actionability (a specific next step).
- Track cost and latency per call; cache explanations until the deal's score changes.

APIs, dashboard and infrastructure

One Go API serves the dashboard and the MCP server; everything runs with `make up` on Docker Compose and ships to Kubernetes with Helm.

REST API (Go, OpenAPI spec checked in)

Endpoint	Returns
`GET /v1/deals?team=&risk=high&limit=`	Paged deals with p_win, slip_risk, health summary
`GET /v1/deals/{id}`	Deal detail: score history, features, recent events
`GET /v1/deals/{id}/explanation`	Agent explanation and next action (cached)
`GET /v1/forecast?team=&period=`	Expected bookings with P10/P90 and contributing deals
`POST /v1/sync/salesforce`	Triggers a CRM pull (admin only)
`GET /healthz`, `GET /metrics`	Liveness and Prometheus metrics

Add a thin GraphQL layer only as a stretch goal, to match Outreach's frontend stack. Use API-key auth, per-key rate limiting and cursor pagination.

Dashboard (React + TypeScript + Vite)

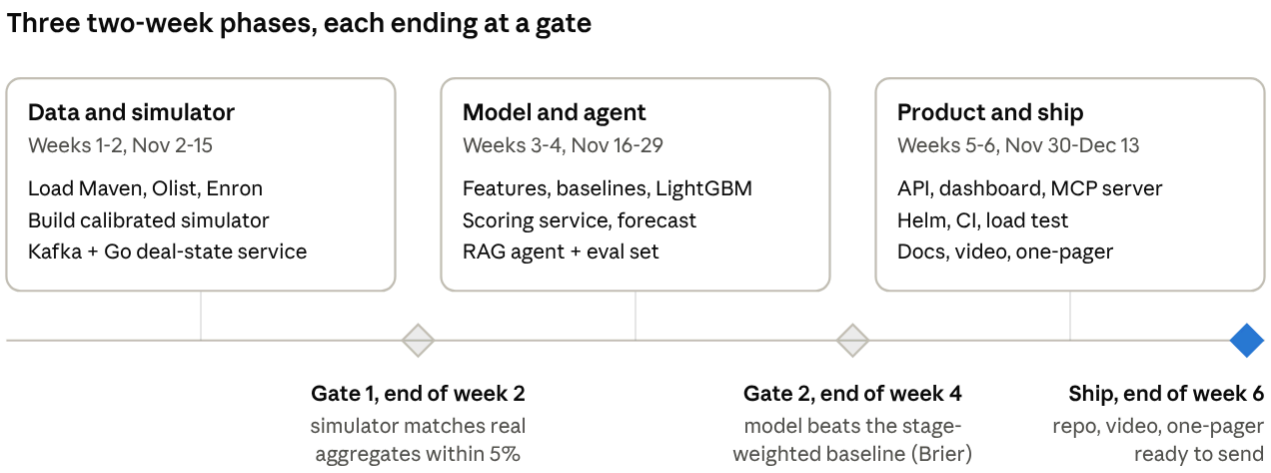
- **Forecast view:** expected bookings for the quarter with the P10–P90 band, vs the stage-weighted baseline.
- **At-risk list:** deals sorted by risk × amount, with a one-line reason each.
- **Deal page:** score over time, activity timeline, top factors, and an "Explain" button calling the agent.
- **Live mode:** scores update over Server-Sent Events while the simulator streams.

Infrastructure

- **Local:** Docker Compose with Redpanda (Kafka-compatible, lighter), Postgres + pgvector, Redis, MLflow, Prometheus, Grafana.
- **Cluster:** Helm chart for kind/minikube; optional deploy to a small AWS EKS or a single EC2 box for the demo. Terraform for any cloud resources.
- **CI (GitHub Actions):** lint, unit tests, integration tests with Testcontainers, build images, run a short end-to-end simulation.
- **Observability:** Prometheus metrics (consumer lag, events/s, scoring latency, API p95, LLM cost), Grafana dashboards, OpenTelemetry tracing from event to score, structured JSON logs.

Timeline

Six weeks from Monday 2 November to 13 December 2026, in three phases; do not start a phase until the previous gate passes.



6-week plan · 3 phases, 3 gates

If a gate slips, cut scope from the next phase (see Risks), not from the gate's quality bar.

Week 1 — data foundation

- [ ] Repo, Makefile, Docker Compose with Redpanda, Postgres, Redis
- [ ] Loaders and cleaning for Maven, Olist and Enron; common `deals` table
- [ ] Fit real distributions: cycle length, win rate, deal size, reply delay
- [ ] Salesforce Developer org set up with the deals loaded; request Outreach developer access

Week 2 — simulator and stream

- [ ] Simulator v1: latent health, Poisson event rates, outcome-linked patterns, noise
- [ ] Validation report: aggregates within 5%, KS test on reply delay, leakage check
- [ ] Stream mode into Kafka; Go deal-state service with idempotent consumer
- [ ] Go CRM connector pulling from Salesforce

Week 3 — model

- [ ] Point-in-time feature library shared by training and scoring
- [ ] Stage-weighted and logistic baselines; LightGBM with isotonic calibration
- [ ] Time-based evaluation, snapshot scoring, ablations; MLflow tracking

Week 4 — scoring, forecast, agent

- [ ] Python scoring service on `deal.features` → `deal.scores`
- [ ] Monte Carlo team forecast with P10/P90

- [ ] Playbook written and indexed in pgvector; Deal Risk Agent with grounding guard
- [ ] 50-case agent eval set and first scores

Week 5 — product and infrastructure

- [ ] Go REST API with SSE; MCP server with 5 tools
- [ ] React dashboard: forecast, at-risk list, deal page, Explain
- [ ] Helm chart, GitHub Actions CI, Prometheus and Grafana dashboards
- [ ] Load test to 1M events/hour; fix bottlenecks

Week 6 — polish and pitch

- [ ] Results report, design doc, ADRs, postmortem log
- [ ] README with GIF; fresh-clone `make up` test on another machine
- [ ] 3-minute demo video and one-page summary
- [ ] Send to Outreach India engineering leaders; add to resume

Repo structure, testing and documentation

One monorepo, so a reviewer clones once and runs everything.

```
pipeline-pulse/
  services/
    connector/      # Go: Salesforce/HubSpot pull -> activity.events
    deal-state/     # Go: consumes events, maintains deal_health, emits
deal.features
    api/            # Go: REST + SSE, serves dashboard and MCP server
  ml/
    data/           # loaders for Maven, Olist, Enron + cleaning
    simulator/      # Python: calibrated activity generator (batch + stream)
    features/       # point-in-time feature code shared by training and
scoring
    training/       # baselines, LightGBM, calibration, evaluation notebooks
    scoring/        # Python: consumes deal.features, emits deal.scores
  agent/            # Deal Risk Agent, RAG indexing, MCP server, eval set
  web/              # React + TypeScript dashboard
  deploy/           # docker-compose, Helm chart, Terraform, Grafana
dashboards
  docs/             # design doc, ADRs, results report, demo script
  Makefile          # make up | make seed | make train | make demo | make
test
```

Testing

Layer	What to test	Tools
Unit	Feature functions (point-in-time correctness), health updates, API handlers	Go `testing`, pytest
Property	Simulator invariants: no events after close, stage order respected	Hypothesis
Integration	Event → deal_health → score through real Kafka and Postgres	Testcontainers
Idempotency	Replaying the same 10k events gives identical state	Custom replay test
Load	1M events/hour; consumer lag stays bounded	k6 or a Go load generator
Model	Metrics regression: CI fails if Brier gets worse than the last release	pytest + MLflow
Agent	Groundedness on the 50-case eval set	Custom eval script

Documentation

- **README:** the pitch in 3 lines, a GIF of the dashboard, `make up`, and results.
- **Design doc (2–3 pages):** problem, architecture, tradeoffs, what I would do with real data.
- **ADRs:** one page per big decision (for example "Kafka over a database queue", "LightGBM over a deep model", "pgvector over a separate vector DB").
- **Results report:** metrics tables, calibration plot, ablations, failure cases.
- **Postmortem log:** what broke during the build and how I fixed it — strong interview material.

Risks and mitigations

The biggest risk is a model that only learns the simulator's own rules; the leakage and ablation checks exist to catch that.

Risk	Impact	Mitigation
Model learns simulator rules, not sales behaviour	Results look great but mean nothing	15% pattern-breaking noise, leakage check, report real-only vs simulated ablation honestly

Risk	Impact	Mitigation
Maven data is fictional and small (~8.8k deals)	Weak claims about realism	Use Olist (real) as a second holdout; say plainly which data is real
Scope creep across 7 components	Nothing finished by week 6	Cut order: GraphQL → HubSpot → Kubernetes → survival model; never cut evaluation or docs
Outreach sandbox access not granted	No real Outreach connector	Mirror its public API objects; Salesforce connector proves real integration
LLM cost or rate limits during demo	Demo fails live	Cache explanations; offline fallback with a local model; recorded demo video
Kafka/Kubernetes setup eats time	Delays ML work	Start on Redpanda in Compose; add Helm only in week 5
Pitch reads as "your product is broken"	Bad first impression	Frame it as an opportunity backed by public reviews; state assumptions and limits

Pitch package and JD skill mapping

Leadership reads the one-pager and watches the video; engineers open the repo — so build all three, each standing on its own.

The package

- **One-page summary:** the gap, what I built, 3 headline numbers, the honest limits, one ask (a conversation).
- **3-minute demo video:** the simulator streaming → scores moving live → "Explain" on an at-risk deal → the forecast vs baseline.
- **GitHub repo:** README with a GIF and `make up`, design doc, ADRs, results report.
- **Resume line:** "Built Pipeline Pulse, an event-driven deal-risk and forecasting engine (Go, Kafka, LightGBM, LLM agent over MCP); X% lower Brier score than stage-weighted forecasting at 1M events/hour."

Where to use it

- The campus pre-placement talk Q&A and the interview "tell me about a project" question.
- A short LinkedIn note to Outreach India engineering leaders with the video link — three sentences, no attachment.
- The "why Outreach" answer: you studied the market and built toward their direction.

JD requirement → evidence in the project

JD asks for	Evidence
Distributed systems, large-scale software	Kafka pipeline with partitioning by deal, idempotent consumers, replay, 1M events/hour load test
Cloud computing	Docker, Helm on Kubernetes, optional AWS deploy with Terraform
Data platforms	Event schema design, append-only log, point-in-time feature pipeline
AI/ML	Calibrated LightGBM, honest evaluation and ablations, RAG agent with a groundedness eval, MCP
Python, C/C++ or similar	Python for ML and the simulator; Go for services (Outreach's main backend language)
Software architecture, design patterns	Service boundaries, provider interface for LLMs, strategy pattern for models, ADRs
Problem solving	Simulator calibration and the leakage check; postmortem log
Full lifecycle, operational excellence	CI/CD, tests at every layer, metrics, tracing, dashboards
Documentation	Design doc, ADRs, results report, README
Communication	One-pager and demo video aimed at leadership